

1. Análisis de Complejidad Computacional "Insertion Sort Advanced Analysis"

1.1. Introducción

La solución implementada para el problema *Insertion Sort Advanced Analysis* calcula el número de inversiones presentes en un arreglo. Una inversión corresponde a un par de elementos que se encuentran en un orden contrario al esperado en un arreglo ordenado.

La implementación utiliza dos ciclos anidados para comparar cada elemento con todos los elementos ubicados a su derecha y determinar si existe una inversión.

1.2. Análisis de la Complejidad

La estructura principal del algoritmo es la siguiente:

```
for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] > arr[j]:
            shifts += 1
```

Sea n el número de elementos del arreglo.

El ciclo externo se ejecuta n veces. Para cada iteración del ciclo externo, el ciclo interno realiza una cantidad decreciente de comparaciones. Cuando $i = 0$ se realizan $n - 1$ comparaciones, cuando $i = 1$ se realizan $n - 2$ comparaciones, y así sucesivamente hasta llegar a una única comparación.

Por lo tanto, el número total de operaciones ejecutadas por el algoritmo puede expresarse mediante la siguiente sumatoria:

$$T(n) = \sum_{i=0}^{n-1} (n - i - 1)$$

Desarrollando la expresión:

$$T(n) = (n - 1) + (n - 2) + (n - 3) + \cdots + 2 + 1$$

Esta expresión corresponde a la suma de los primeros $n - 1$ números naturales:

$$T(n) = \sum_{k=1}^{n-1} k$$

Aplicando la fórmula de la suma de una progresión aritmética:

$$\sum_{k=1}^m k = \frac{m(m + 1)}{2}$$

y sustituyendo $m = n - 1$:

$$T(n) = \frac{(n-1)n}{2}$$

Desarrollando el producto:

$$T(n) = \frac{n^2 - n}{2}$$

Para determinar la complejidad asintótica se conservan únicamente los términos dominantes, ignorando constantes multiplicativas y términos de menor orden:

$$T(n) \in O(n^2)$$

Por consiguiente, la complejidad temporal del algoritmo es:

$$\boxed{O(n^2)}$$

1.3. Conclusion

El análisis mediante sumatorias permite demostrar formalmente que el número total de comparaciones realizadas por el algoritmo corresponde a:

$$\frac{n(n-1)}{2}$$

lo que conduce a una complejidad temporal cuadrática $O(n^2)$. Por lo que su crecimiento cuadrático hace que el tiempo de ejecución aumente considerablemente cuando el tamaño de la entrada es demasiado grande.